DATABRICKS ASSOCIATE EXAM

Comprehensive Study Guide

Introduction to Data Engineering in Databricks

Chapters 1–2: Data Ingestion with Lakeflow Connect | Orchestration with Lakeflow Jobs

---

TECHNICAL TERM GLOSSARY

Master these definitions — they appear throughout both exam chapters and are frequently tested.

Idempotency — An operation that produces the same result no matter how many times it is executed. COPY INTO is idempotent — it skips files already loaded, so re-running it never creates duplicate records. CTAS is NOT idempotent because it re-reads all data every run.
ACID Transactions — Atomicity, Consistency, Isolation, Durability. Delta Lake guarantees all four properties, enabling safe concurrent reads/writes without data corruption.
Medallion Architecture (Multi-Hop) — A data organisation pattern with three quality layers: Bronze (raw ingestion), Silver (cleaned/enriched), Gold (business-aggregated). Data progressively improves as it moves through each layer.
DAG (Directed Acyclic Graph) — A conceptual structure representing a series of tasks with directional, non-circular dependencies. Directed = each edge has one direction. Acyclic = no circular loops. Graph = vertices (tasks) connected by edges (dependencies).
DML (Data Manipulation Language) — SQL commands that modify data: INSERT, UPDATE, DELETE, MERGE.
Schema Evolution — The ability to add or modify columns in a table over time without breaking existing workflows. Delta Lake and Auto Loader both support schema evolution.
Schema Enforcement — A guarantee that only data matching the defined schema can be written to a table. Rejects non-conforming records.
Time Travel — A Delta Lake feature allowing you to query a previous version of a table using VERSION AS OF or TIMESTAMP AS OF syntax.
Incremental Ingestion — Loading only new or changed data since the last pipeline run, rather than reprocessing the entire dataset. More efficient than full batch.
Streaming Table — A table registered in Unity Catalog with built-in support for incremental or streaming data processing. Automatically backed by a Lakeflow Declarative Pipelines pipeline.
Unity Catalog — Centralised data catalog providing access control, auditing, data lineage, quality monitoring, and data discovery across all Databricks workspaces.
DBU (Databricks Unit) — The billing unit used by Databricks. Different compute types consume DBUs at different rates.
Photon — Databricks-native query engine written in C++, enabled by default on Serverless compute. Delivers significant query performance improvements over standard Spark at no extra configuration cost.
CDC (Change Data Capture) — A pattern for tracking and capturing data changes (inserts, updates, deletes) in a source system so they can be replicated incrementally to a target.

SCD Type 1 — Slowly Changing Dimension pattern where old data is overwritten — no history is preserved. MERGE INTO is ideal for SCD Type 1.
SCD Type 2 — Slowly Changing Dimension pattern where both old and new records are retained by adding a new row with a version/effective date column.
Checkpoint (Structured Streaming) — A directory where Spark Structured Streaming saves its progress metadata, enabling fault-tolerant restart and exactly-once processing.
Dynamic Value Reference — A runtime placeholder in Lakeflow Jobs using {{ }} notation (e.g. {{job.start_time.day}}) that resolves to actual values during execution.
Task Values — Key-value pairs set and retrieved between tasks during a job run using dbutils.jobs.taskValues.set() and .get(). Computed at runtime — unlike parameters which are set before execution.
Repair & Rerun — A Lakeflow Jobs feature allowing selective re-execution of only the failed tasks in a job run, saving time and compute costs.
VARIANT Data Type — A new Databricks data type (Public Preview as of 2025 Q2) that can store any data including JSON. Highly flexible and more performant than storing JSON in STRING columns.
Rescued Data Column (_rescued_data) — A special column automatically added during ingestion when source data does not match the target schema. Mismatched values are stored as JSON strings here instead of being dropped.
_metadata Column — A hidden column available for all input file formats during ingestion, providing file-level metadata such as file_name and file_modification_time.
Serverless Compute — Fully managed compute provided by Databricks in the Databricks account. Eliminates VM selection, capacity planning, and infrastructure maintenance. Photon enabled by default.
Lakeflow — Databricks's end-to-end data engineering solution. Has three components: Lakeflow Connect (ingestion), Lakeflow Declarative Pipelines (ETL), and Lakeflow Jobs (orchestration).
Lakehouse Federation — A feature enabling direct SQL queries against external data sources (e.g. MySQL, Snowflake) without copying data into the Lakehouse. Useful for ad-hoc queries and proof-of-concept work.
Zerobus — A Lakeflow Connect API for writing event data directly to the Lakehouse at very high throughput (100 MB/s) with near real-time latency (<5 seconds). Ideal for IoT, clickstreams, and telemetry.
Delta Sharing — Open protocol for securely sharing Delta Lake data across platforms, clouds, and organisations without data copying.
Databricks Marketplace — Open exchange for data products (datasets, notebooks, dashboards, ML models, solution accelerators) powered by Delta Sharing.

CHAPTER 1: DATA INGESTION WITH LAKEFLOW CONNECT

Course: Introduction to Data Engineering in Databricks
Official Databricks Lakeflow Documentation

1.1 What is Lakeflow Connect?

Traditionally, organisations rely on a patchwork of third-party tools and custom in-house solutions to move data from different sources (On-Prem files, legacy systems, cloud storage, databases, SaaS applications, real-time event streams) into a central data platform. Lakeflow Connect solves this fragmentation.

Definition

Lakeflow Connect is the ingestion component of Databricks Lakeflow. It provides simple, efficient, built-in connectors to bring data from files, cloud storage, databases, enterprise applications, and streaming sources directly into the Databricks Lakehouse — all within a unified, managed platform.

Key Benefits

- Managed and efficient solution — lower costs and quicker time to value
- Self-serve interfaces for every practitioner — democratised data ingestion
- Unified observability and governance — secured, healthy pipelines and tables
- Simple setup and maintenance with no custom integration code required

Architecture: Lakeflow Connect is all Ingestion

Data Sources supported: On-prem, Databases, Cloud Storage, SaaS Applications, Event Streams.
Output: feeds directly into the Databricks Data Intelligence Platform for Streaming Analytics, Data Science & ML, Data Sharing, and BI & Reporting.

1.2 Three Connector Types

Connector Type	Description	Best For
Upload Files	Upload local files directly to Databricks as a Volume or table	Small, one-off file loads
Standard Connectors	Cloud Object Storage, Kafka, and other streaming/batch sources	Cloud storage and streaming pipelines
Managed Connectors	Purpose-built for SaaS apps and enterprise databases with efficient incremental reads	Enterprise systems (Salesforce, Workday, PostgreSQL, etc.)

1.3 Ingestion Methods Overview

Ingestion Methods Documentation

Method 1 — Batch (Full Load)

Loads all records every time the pipeline runs. Traditional approach; re-ingests entire dataset.

- SQL: CREATE TABLE AS SELECT (CTAS)
- Python: spark.read.load()

Exam Tip: Batch is NOT idempotent — it always re-reads all data. Do not confuse with COPY INTO.

Method 2 — Incremental Batch

Only new records are ingested; previously loaded records are automatically skipped.

- SQL: COPY INTO
- Python: spark.readStream (Auto Loader with timed trigger)
- Declarative Pipelines: CREATE OR REFRESH STREAMING TABLE

Key Property: COPY INTO is retryable and idempotent — safe to re-run without duplicates.

Method 3 — Streaming (Continuous / Micro-batch)

Data is continuously loaded as it is generated, enabling near real-time query.

- Sources: Apache Kafka, Amazon Kinesis, Google Pub/Sub, Apache Pulsar
- Python: spark.readStream (Auto Loader with continuous trigger)
- Declarative Pipelines: trigger mode continuous

1.4 Delta Lake Review

Delta Lake Documentation

What is Delta Lake?

Delta Lake is an open-source protocol for reading and writing files to cloud storage. Delta tables are the open table format that underpins the Lakehouse architecture.

Delta Table Physical Structure

A Delta table is a folder in cloud storage containing two things:

- Parquet files — the actual data
- _delta_log/ folder — JSON transaction logs (metadata) tracking all changes

The transaction log is what enables ACID guarantees, time travel, and schema enforcement.

Key Delta Lake Features

- ACID Transactions — atomic, consistent, isolated, durable operations for concurrent workloads
- DML Support — INSERT, UPDATE, DELETE, MERGE on cloud data files

- Time Travel — query previous table versions with VERSION AS OF or TIMESTAMP AS OF
- Schema Evolution & Enforcement — flexible but governed schema management
- Unified Batch & Streaming — same table used for both batch and streaming workloads
- Open Source — integrates with Parquet and Iceberg ecosystems

Delta Lake Key Features

1.5 Medallion Architecture (Multi-Hop)

Medallion Architecture Overview

Layer	Purpose	Data State
Bronze	Raw data ingestion — "as-is" from source	Raw, unprocessed, unvalidated
Silver	Cleaned, transformed, and enriched data	Validated, deduplicated, joined
Gold	Business-level aggregations optimised for analytics	Curated, aggregated, BI-ready

Data flows: Ingest (batch or streaming) → Bronze → Silver → Gold → Consumers (BI & Reporting, ML & AI, Streaming Analytics).

1.6 Data Ingestion from Cloud Storage

Cloud Storage Ingestion Documentation

Three primary methods for ingesting CSV, JSON, Parquet (and other formats) from cloud object storage into Delta tables:

(1 of 3) CREATE TABLE AS (CTAS)

Creates a Delta table by default from files in cloud object storage using the read_files() function.

```
CREATE TABLE new_table AS
SELECT *
FROM read_files(
  '<path_to_file(s)>',
  format => '<file_type>',
  <other_format_specific_options>
);
```

read_files() Capabilities

- Supports JSON, CSV, XML, TEXT, BINARYFILE, PARQUET, AVRO, ORC
- Automatically detects file format and infers a unified schema across all files
- Allows format-specific options for greater control
- Can be used in streaming tables to incrementally ingest via Auto Loader

Property	Value

Ingestion Type	Batch (full load)
Idempotent?	No — re-reads all data each run
Use Case	Small datasets, one-time or ad hoc ingestion, scheduled full reloads
Latency	High
Schema Evolution	Manual or inferred during read

(2 of 3) COPY INTO (Incremental Batch — Legacy)

```
CREATE TABLE new_table;
```

```
COPY INTO new_table
FROM '<dir_path>'
FILEFORMAT = <file_type>
FORMAT_OPTIONS(<options>)
COPY_OPTIONS(<options>)
```

COPY INTO Key Properties

- Retryable and idempotent — skips files already loaded
- Supports Parquet, JSON, XML, CSV, and others
- FORMAT_OPTIONS() — controls how source files are parsed
- COPY_OPTIONS() — controls behaviour such as schema evolution (mergeSchema) and idempotency override (force)
- FROM clause — path to cloud storage location where new files arrive

Property	Value
Ingestion Type	Incremental Batch
Idempotent?	Yes — skips previously loaded files
Use Case	Thousands of files, scheduled incremental loads, simple SQL-based pipelines
Latency	Moderate (scheduled)
Schema Evolution	Supported with options

(3 of 3) Auto Loader

Auto Loader Documentation

Auto Loader incrementally and efficiently processes new data files as they arrive in cloud storage — in either batch or streaming mode — without any additional setup.

Python (Structured Streaming)

```
(spark
  .readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "<checkpoint_path>")
  .load("/Volumes/catalog/schema/files")
  .writeStream
```

```
  .option("checkpointLocation", "<checkpoint_path>")
  .trigger(processingTime="5 seconds")
  .toTable("catalog.database.table")
)
```

SQL (Declarative Pipelines)

```
CREATE OR REFRESH STREAMING TABLE catalog.schema.table
SCHEDULE EVERY 1 HOUR AS
SELECT * FROM STREAM read_files(
  '<dir_path>',
  format => '<file_type>'
)
```

Property	Value
Ingestion Type	Incremental Batch or Streaming
Idempotent?	Yes
Use Case	Millions+ files/hour, backfill of billions of files, near real-time streaming
Latency	Low or high (configurable)
Schema Evolution	Automatic — detects and evolves schemas with new columns

Full Comparison: CTAS vs COPY INTO vs Auto Loader

Feature	CTAS + spark.read	COPY INTO	Auto Loader
Type	Batch	Incremental Batch	Inc. Batch or Streaming
Idempotent	No	Yes	Yes
Schema Evolution	Manual/inferred	With options	Automatic
Latency	High	Moderate	Low or high
Best For	Ad hoc, small datasets	Thousands of files, scheduled	Millions of files, near real-time

1.7 Appending Metadata Columns on Ingest

File Metadata Column Documentation

When ingesting files from cloud storage, you can add file-level metadata to each row using the special _metadata hidden column. This preserves context about data origin — critical for auditing, lineage, and debugging.

Common _metadata Fields

- _metadata.file_modification_time — last modification timestamp of the source file
- _metadata.file_name — name of the source file for each row

Usage

```
SELECT *, _metadata.file_name, _metadata.file_modification_time
FROM read_files('/path/to/files', format => 'csv')
```

Exam Tip: _metadata must be explicitly selected in your query — it is not included automatically.

1.8 Working with the Rescued Data Column

During ingestion (via read_files(), spark.read, or Auto Loader), if the raw data does not match the target table schema, columns that do not match are not dropped — they are rescued.

How It Works

- Non-conforming values are stored as JSON strings in the _rescued_data column
- Conforming values are stored normally in their respective columns
- _rescued_data is null for rows where all values matched the schema

Example

If the target schema expects cost as BIGINT but the source has "$100" (a STRING):

- The cost column for that row will be null in the Delta table
- The _rescued_data column will contain: {"cost": "$100", "_file_path": "<file_path>"}

Exam Tip: COPY INTO does NOT provide a rescued data column — only CTAS (read_files), Auto Loader, and spark.read do.

1.9 Ingesting Semi-Structured Data: JSON

JSON Structure Review

JSON data consists of key-value pairs enclosed in curly braces {}. Values can be: String, Number, Boolean, Array, Object (nested JSON).

Working with JSON Columns: Three Approaches

Approach 1: STRING Data Type

- JSON stored as a plain STRING — no schema constraint
- Access sub-fields using colon (:) syntax: SELECT json_column:name, json_column:address:city
- Flexible but less performant

Approach 2: STRUCT Data Type (Recommended)

- Parse JSON into a STRUCT with a defined schema — enforces structure
- More efficient for querying than raw STRING
- Step 1: Use schema_of_json() to auto-derive the schema from a sample JSON string

```
SELECT schema_of_json('{"name": "John", "age": 35, "address": {...}}')
```

- Step 2: Use from_json() to convert the STRING column to STRUCT

```
SELECT from_json(json_col, '<json-struct-schema>') AS struct_column FROM table
```

JSON Type Mapping to Databricks SQL Types

JSON Type	Databricks SQL Type
String	STRING
Number	INT / FLOAT / DOUBLE
Boolean	BOOLEAN
Object	STRUCT<>
Array	ARRAY<>

Approach 3: VARIANT Data Type (Public Preview as of 2025 Q2)

- Can store any data type including JSON
- Highly flexible — adapts to different data shapes without rigid schemas
- Improved performance over existing STRING/STRUCT methods

Exam Tip: VARIANT is in Public Preview. Watch for its General Availability (GA) announcement.

1.10 Ingesting into Existing Delta Tables: MERGE INTO

MERGE INTO Documentation

MERGE INTO applies updates, insertions, and deletions from a source table into an existing Delta table in a single atomic operation.

Full MERGE INTO Syntax

```
MERGE INTO target_table target
USING source_table source
ON target.id = source.id
WHEN MATCHED AND source.status = 'update' THEN
  UPDATE SET target.email = source.email, target.status = source.status
WHEN MATCHED AND source.status = 'delete' THEN
  DELETE
WHEN NOT MATCHED THEN
  INSERT (id, email, status) VALUES (source.id, source.email, source.status)
```

MERGE INTO Clause Logic

Clause	When It Fires	Action
WHEN MATCHED	Row exists in both source and target	UPDATE or DELETE
WHEN NOT MATCHED	Row exists in source but NOT in target	INSERT
WHEN NOT MATCHED BY SOURCE	Row exists in target but NOT in source	UPDATE or DELETE

Use Cases for MERGE INTO

- Slowly Changing Dimensions (SCD Type 1) — overwrite matching rows
- Incremental loads with CDC (Change Data Capture)
- Complex change data capture scenarios with insert/update/delete in one pass

Exam Tip: MERGE INTO supports schema enforcement OR schema evolution (configured via options).

1.11 Ingesting Enterprise Data: Lakeflow Connect Managed Connectors

Managed Connectors — SaaS
Managed Connectors — Database

For enterprise databases and SaaS applications, Lakeflow Connect Managed Connectors provide a fully managed, low-code ingestion experience with efficient incremental read/write patterns.

Supported Enterprise Sources (examples)

- Workday, Salesforce, Google Analytics, ServiceNow, Microsoft Dynamics 365
- PostgreSQL, Oracle, SQL Server, MySQL (and other relational databases)

SaaS Connector Architecture

1. A Lakeflow Serverless Declarative Pipelines job collects credentials from Unity Catalog
2. The job reaches out to the publicly accessible data source (e.g. API, open OLAP port)
3. The service transforms the data and stores it in a Streaming Delta Table

Database Connector Architecture — Two Additional Components

- Ingestion Gateway — a dedicated Classic Compute pipeline that connects to the database to extract metadata, snapshots, and change logs, staging them in a Unity Catalog Volume
- Unity Catalog Volume — intermediate staging area between the gateway and the managed ingestion pipeline. Secured by standard UC mechanics.

The Ingestion Gateway exists because many customer databases sit behind firewalls or are not publicly accessible — the gateway can be deployed inside the customer network.

1.12 Partner Connect

Partner Connect Documentation
Databricks Partner Connect Portal

When no managed connector exists for a specific data source, Partner Connect allows you to create trial accounts with select technology partners and connect Databricks to partner solutions directly from the Databricks UI.
Example partners: Informatica, Qlik, Rivery, Alteryx, Prophecy, Fivetran.

1.13 Additional Ingestion Features (Beyond Course Scope)

Lakehouse Federation

Lakehouse Federation Documentation
Query external data sources directly without moving data into the Lakehouse. Best for ad-hoc reporting, proof-of-concept work, exploratory ETL, and hybrid migration workloads.

Zerobus

Zerobus at Data + AI Summit
A Lakeflow Connect API for writing event data directly to the Lakehouse at very high throughput (100 MB/s) with near real-time latency (<5 seconds). Designed for IoT, clickstreams, and telemetry use cases.

Delta Sharing

Delta Sharing Documentation
Open protocol for securely sharing live Delta Lake data across platforms, clouds, and organisations without data copying.

Databricks Marketplace

Databricks Marketplace Documentation
Databricks Marketplace Portal
An open exchange for all data products: Datasets, Notebooks, Dashboards, ML Models, and Solution Accelerators. Powered by Delta Sharing.

Chapter 1 — Key Learning Outcomes

- Describe Lakeflow Connect as a solution for simple and scalable data ingestion
- Explain the benefits of Delta tables and the Medallion architecture
- Describe how to ingest files from cloud object storage using CREATE TABLE AS and COPY INTO
- Explain how to add file metadata columns (_metadata) in Bronze layer tables
- Explain how to leverage rescued columns (_rescued_data) for data quality
- Explain how to ingest and flatten semi-structured data like JSON files
- Describe options to ingest data from enterprise systems using Managed Connectors
- Describe how MERGE INTO can be used during ingestion for CDC/SCD patterns
- Understand how Auto Loader and SQL TVFs (read_files) can be used for ingestion

CHAPTER 2: ORCHESTRATION WITH LAKEFLOW JOBS

Course: Introduction to Data Engineering in Databricks

2.1 Data Engineering in Databricks — The Full Stack

The Databricks data engineering stack has four layers:

4. Optimised Storage: Delta Lake, Parquet, Iceberg
5. Unified Governance: Unity Catalog — access control, auditing, lineage, quality monitoring
6. Industry-Leading Data Processing Engine: Apache Spark + Structured Streaming
7. Lakeflow — end-to-end data engineering solution with three components:
    - Lakeflow Connect — efficient ingestion connectors
    - Lakeflow Declarative Pipelines (previously DLT) — accelerated ETL development
    - Lakeflow Jobs (previously Workflows) — reliable orchestration for analytics and AI

Chapter 2 focuses specifically on Lakeflow Jobs — the orchestration layer.

2.2 What is Lakeflow Jobs?

Lakeflow Jobs is a workflow automation tool for Databricks that orchestrates data processing workloads. External orchestration tools (Apache Airflow, Prefect, Dagster, dbt) are not unified with the Lakehouse — they create integration challenges, data silos, and higher costs.

Problems Solved by Lakeflow Jobs

- Hard to use for many practitioners → data teams become less productive
- Difficult to understand root cause when issues occur → bad data quality
- Complex architecture to manage → higher cost of ownership and lower reliability

Lakeflow Jobs Architecture

Component	Description
Workflow Engine	Central coordinator that manages task execution order and dependencies
Compute Layer	Supports ETL, ML/AI, and Analytics/BI workloads
Triggers	Scheduled, Continuous, File Arrival, Table Update, Manual
Observability	Built-in monitoring and troubleshooting
Control Flow	Manages task dependencies and execution order

2.3 Building Blocks of Lakeflow Jobs

Job

A Job is the primary resource for scheduling, coordinating, and running operations such as data processing, ETL, analytics, and machine learning workloads. Think of a Job as the container for your entire workflow.

Task

A Task is a single unit of work within a Job that executes a specific workload. Each Job consists of one or more Tasks — they are the building blocks that do the actual work.

Supported Task Types

- Databricks Notebooks (any language)
- Lakeflow Declarative Pipelines (DLT)
- SQL Files and Queries
- Python Scripts
- Python Wheels (packaged code)
- AI/BI Dashboards
- Spark Submit (legacy Spark applications)
- dbt (data transformation)
- Java JAR files
- Power BI

Supported Languages

- Python, SQL, Scala, R, Java (via JAR file)

2.4 Compute Options for Jobs

Compute Types Documentation

Compute Type	Best For	Key Considerations
Interactive Clusters	Ad-hoc analysis, data exploration, development	NOT for production — costly, limited scalability, availability issues from shared use
Job Clusters	Production workloads	~50% cheaper (terminate when job ends), subject to cloud provider startup times
Serverless	Most production workloads	Fully managed, Photon default, faster clusters, auto-scaling, lower TCO, shielded from cloud disruptions

SQL Warehouse	SQL queries, dashboards, BI	Serverless by default, high concurrency, Intelligent Workload Management, auto-start/stop

Compute Selection Principle

Each Task within a Job can be assigned its own compute resource. Tasks in the same Job can share the same compute or use different compute — optimising both performance and cost per task type.

Serverless Performance Mode

Mode	Description	Best For
Standard (Performance Optimized OFF)	Focuses on cost-efficiency. Longer startup (4–6 minutes).	Non-urgent workloads with flexible timing
Optimized (Performance Optimized ON)	Faster job startup and execution.	Time-sensitive workloads

Exam Tip: Performance Optimized mode applies ONLY to tasks using Serverless compute.

2.5 Task Orchestration and DAGs

What is a DAG?

A DAG (Directed Acyclic Graph) is a conceptual representation of a series of activities including data processing flows.

- Directed — unambiguous direction for each edge (task flow)
- Acyclic — contains no cycles (no Task A → Task B → Task A loops)
- Graph — collection of vertices (tasks) connected by edges (dependencies)

Orchestration Tools

- Databricks UI — visual drag-and-drop DAG builder
- Databricks API — programmatic job creation and management
- Databricks SDK — language-specific libraries (Python, Java, Go)
- Databricks Asset Bundles (DABs) — GitOps-style infrastructure-as-code

Common Workflow Patterns

Pattern	Shape	Use Case
Sequence	Linear chain	Data transformation, cleaning, Bronze → Silver → Gold
Funnel	Multiple sources → one target	Data collection and consolidation from multiple sources
Fan-out (Star)	One source → multiple targets	Data ingestion and distribution to multiple downstream systems

2.6 Common Task Configuration Options

1. Parameters

Parameters are key-value pairs that pass values into tasks. They exist at two levels:

Task Parameters

- Defined at the individual task level
- Access in Notebooks: dbutils.widgets.get("parameter_name")
- Specific to each task — fine-grained control

Job Parameters

- Defined at the job level — automatically pushed down to ALL tasks
- Override Task Parameters when the same key exists
- Can be overridden at runtime when triggering job runs
- Access in Notebooks: dbutils.widgets.get("parameter_name") — same syntax, system handles precedence

Critical Rule: Job Parameters override Task Parameters when the same key exists.

2. Task Values (Dynamic Inter-Task Communication)

Task Values are computed at runtime and enable tasks to share results with downstream tasks.

Setting Task Values

```
dbutils.jobs.taskValues.set(
  key = "catalog_name",
  value = "dbacademy"
)
```

Retrieving Task Values from Another Task

```
dbutils.jobs.taskValues.get(
  taskKey = "upstream-task-name",
  key = "catalog_name"
)
```

3. Dynamic Value References ({{ }} Notation)

Dynamic Value References allow referencing values at runtime from job and task context. They use double-brace notation.

Reference	Description
{{job.start_time.day}}	Access execution timing for date-based processing
{{job.run_id}}	Unique identifier for tracking and logging
{{job.parameters.environment}}	Access job-level parameters dynamically
{{task.name}}	Useful for logging and dynamic path generation
{{task.retry_count}}	Track retry attempts for debugging

{{tasks.<task-name>.values.record_count}}	Access computed results from upstream tasks
{{tasks.<task-name>.values.output_path}}	Use dynamic paths generated by other tasks

4. Notification Alerts

Alerts keep teams informed and can be configured at two levels:

- Job Level — alert sent after successful completion of the entire job
- Task Level — alert sent when a specific task completes or fails

Supported destinations: Email, Microsoft Teams, PagerDuty, Slack, Webhook.
Notification triggers: Task begins, completes, fails. Also for Late Jobs (duration threshold exceeded) and Streaming Backlog.

5. Retry Policy

A policy that determines when and how many times failed task runs are automatically retried.

- Configure max retries and retry interval between attempts
- Consider failure type (transient network vs data quality) when designing retry policy
- Aggressive retries can cause cluster resource contention for compute-intensive tasks

2.7 Job Schedules and Triggers

What is a Trigger?

A trigger is a rule that automatically starts a job run based on a specific condition or schedule. Triggers enable automation so jobs can run without manual intervention.

Trigger Types

1. Scheduled Trigger (Cron)

- Runs jobs automatically at set times or intervals (hourly, daily, etc.)
- Uses cron expression syntax for complex scheduling (e.g. "every 15 minutes during business hours")
- Best for: Daily ETL, weekly reports, monthly aggregations, hourly streaming checkpoints

Example: "Every Monday at 6 AM" → 0 6 * * 1 in cron syntax

2. File Arrival Trigger

- Triggers job when new files are detected in a specified storage location
- Supports: AWS S3, Azure Storage, GCP GCS, and Databricks Volumes
- Enables true event-driven data architectures — process immediately when data arrives
- Best for: Partner data feeds, IoT sensor files, financial trading data, log file processing

3. Continuous Trigger

- Starts a new run as soon as the previous run finishes or fails
- Built-in retry logic automatically managed by Databricks
- Ideal for: Streaming workloads requiring constant processing

- Use cases: Real-time analytics dashboards, fraud detection, IoT processing, CDC streams

Monitoring Note: Continuous jobs require different monitoring — they run indefinitely, not as discrete tasks.

4. Manual Trigger (None)

- Jobs run on demand without any schedule or event
- Can be started from UI ("Run now"), API, CLI, SDK, or Databricks Asset Bundles
- Best for: Ad hoc runs, debugging, one-off backfills
- Can be combined with other trigger types

5. Table Update Trigger (New)

- Automatically starts a job whenever specified source tables are updated
- Monitors for insert, update, delete, or merge changes in one or more tables
- Supports up to 10 tables per trigger
- Works with Unity Catalog-managed Delta/Iceberg tables, materialized views, streaming tables
- Can trigger when ANY table is updated, or wait until ALL tables have been updated

Table Update Trigger — Advanced Options

- Minimum Time Between Triggers — prevents over-triggering during rapid table updates
- Wait After Last Change — delays the job until a set period after the most recent update (ensures entire batch has landed)

2.8 Conditional and Iterative Tasks (Advanced)

Three Advanced Task Types

1. Run-if Conditional Task Dependencies

Controls whether a downstream task runs based on the outcomes of upstream tasks. Supports multiple dependency conditions:

- All succeeded — all upstream tasks completed successfully (traditional)
- At least one succeeded — useful when you have redundant data sources or processing paths
- None failed — task executes even if some upstreams were skipped, as long as none explicitly failed
- Custom conditions — specific combinations of task outcomes

Example: Task-4 depends on Task-2 and Task-3 with "At least one succeeded" — if Task-3 fails but Task-2 succeeds, Task-4 still runs.

2. If/Else Task

Adds boolean conditional logic to workflows based on task results, parameter values, or computed metrics.

- Evaluates expressions using boolean operators: ==, !=, >, >=, <, <=
- Branches into a True path and a False path
- Each branch can contain multiple tasks
- Execution condition: "If none of dependency failed and at least one task executed"

Business Logic Examples:

- Data Quality Gates — branch based on record counts, null percentages, or validation results
- Processing Volume Decisions — use different strategies for large vs. small datasets
- Environment-Specific Logic — execute different tasks based on environment parameters

3. For Each Task

Enables iterative processing — the same nested task is applied to every item in an input array.

- Input array can be a JSON array of values: ["A", "B", "C"]
- Each item is passed as {{input}} to the nested task
- Iterations can run in parallel with configurable concurrency
- Downstream tasks depend on the For Each CONTAINER, not individual iterations

For Each Task Components:

- For Each Container — top-level task managing iteration logic, input array, concurrency, and resource allocation
- Nested Task — the actual work performed for each array item (any task type: notebook, SQL, Python, etc.)

Use Case Examples:

- Geographic processing — process data for each state/region in parallel
- Time period processing — handle different date ranges with the same logic
- Customer segment processing — apply same analysis to different customer groups
- File processing — apply identical transformation to multiple file groups

2.9 Handling Task Failures

Repair and Rerun

The Repair feature allows selective re-execution of only the failed tasks in a job run, instead of restarting the entire workflow. This dramatically reduces recovery time and compute costs.

Repair Workflow

8. Identify the failed task(s) in the job run history
9. Optionally modify the failed task or its parameters
10. Click "Repair run" — only failed tasks and their downstream dependencies are re-executed

Key Benefits

- Running only failed tasks can reduce recovery time by 80–90% in complex pipelines
- Parameter Override Capability — fix configuration issues without rebuilding the job
- Faster Resolution — teams can respond quickly without waiting for entire workflows

Important: Repairing a task run does NOT permanently fix the job definition. If you passed a wrong parameter via Repair, you still need to edit the job definition to prevent future failures.

2.10 Monitoring Jobs Performance

system.lakeflow System Tables

system.lakeflow is a built-in, read-only catalog that automatically logs all job activity across all workspaces in a region.

Key Tables in system.lakeflow

- jobs — basic job metadata and configuration information
- job_tasks — task definitions and configuration details
- job_run_timeline — complete execution history for every job run (sliced hourly using period_start_time and period_end_time)
- job_task_run_timeline — detailed execution history for individual tasks
- pipelines — information about Lakeflow Declarative Pipelines pipelines

Timeline tables slice long-running jobs hourly using period_start_time and period_end_time, enabling accurate duration analysis, concurrency tracking, and SLA measurement.

Spark UI for Performance Optimisation

- Timeline view — highlights task start/end, duration, and overlap to spot bottlenecks
- Task-level details — execution timestamps, duration, cluster/runtime, logs, quick I/O
- Query/code details — full text, run ID, wall-clock split (optimizing/pruning vs executing), files read/written, partitions, and spill details

Actionable Insights from Spark UI

- High Planning Time → improve partitioning strategies or metadata optimisation
- High Execution Time → optimise joins, broadcast strategies, or handle skew fixes
- Resource Bottlenecks → identify memory, CPU, or I/O constraints requiring cluster configuration adjustments

2.11 Lakeflow Jobs in Production: Best Practices

Category 1: Compute & Cost Optimisation

- Use job clusters or serverless compute in production (NOT interactive clusters)
- Reserve interactive clusters strictly for development and ad-hoc analysis
- Enable Photon acceleration on all eligible workloads
- Design workflows to reuse clusters across tasks to reduce startup overhead

Category 2: Orchestration & Modularity

- Break complex pipelines into logical, maintainable modules using the "Run Job" task pattern
- Use multi-task jobs for parallel execution and scalable resource utilisation

- Apply conditional logic using Run-if, If/Else, and For Each tasks
- Limit individual jobs to manageable task counts (recommended maximum: 1000 tasks)

Category 3: Monitoring & Governance

- Use service principals (not personal accounts) for job ownership and data access
- Configure alerts for failures, delays, and completions with appropriate escalation
- Leverage Repair & Run to minimise costs and recovery time when handling failures
- Parameterise tasks for maximum reusability and flexibility across environments

Modular Design: Parent-Child Job Pattern

Modular orchestration breaks complex DAGs into smaller, manageable components:

- Parent Job — orchestrates child jobs, maintaining overall workflow coordination
- Child Jobs — individual modules that can be developed, tested, and deployed independently
- Benefits: maintainability, reusability across multiple parent workflows, team collaboration, independent testing

Git Integration for Production Jobs

Git Integration Documentation

- Jobs support execution of notebooks and code directly from Git repositories
- Prevents unintentional production changes when workspace code is modified
- Simplifies the job definition process with a single source of truth
- Enables CI/CD pipelines through GitHub, GitLab, AWS CodeCommit, and other providers

Git Configuration Steps for a Task

11. Step 1 — Source Configuration: Create tasks with Git provider as the source, specify repository, branch/tag, and authentication credentials
12. Step 2 — Path Configuration: Configure the path to your main notebook under the repository root

Best Practice: Use specific branches or tags for production rather than constantly changing main branches.

Serverless Pricing: Classic vs Serverless TCO

Cost Component	Classic	Serverless
DBUs	Paid to Databricks	Single DBU price (includes infrastructure + operational)
Infrastructure	Paid to cloud provider (VMs, networking, security)	Included in DBU price
Operational	Time spent deploying, automating, maintaining	Included — fully managed
TCO Outcome	Higher total cost of ownership	Lower TCO for most organisations

Chapter 2 — Key Learning Outcomes

- Explain the components of the Databricks data engineering stack (Connect, Pipelines, Jobs)
- Describe the building blocks of Lakeflow Jobs: Jobs, Tasks, and their relationships
- Identify supported task types and languages in Lakeflow Jobs
- Compare compute options and select the right compute for production workloads
- Explain DAG concepts and how task dependencies define execution order
- Describe common workflow patterns: Sequence, Funnel, and Fan-out
- Configure task parameters, job parameters, task values, and dynamic value references
- Explain all five trigger types: Scheduled, File Arrival, Continuous, Manual, Table Update
- Implement advanced control flow using Run-if, If/Else, and For Each task types
- Use Repair & Rerun to recover from task failures efficiently
- Monitor job performance using system.lakeflow system tables and Spark UI
- Apply production best practices for compute, modular design, Git integration, and governance

EXAM QUICK REFERENCE CARD

High-Frequency Exam Topics

Topic	Key Point to Remember
CTAS vs COPY INTO	CTAS = batch, NOT idempotent. COPY INTO = incremental batch, idempotent.
Auto Loader format	.format("cloudFiles") in readStream — enables Auto Loader
_rescued_data	Available in CTAS, Auto Loader, spark.read. NOT in COPY INTO.
_metadata column	Must be explicitly selected. Contains file_name and file_modification_time.
MERGE INTO clauses	MATCHED → UPDATE/DELETE. NOT MATCHED → INSERT. NOT MATCHED BY SOURCE → UPDATE/DELETE.
Job vs Task Parameters	Job Parameters override Task Parameters when the same key exists.
Dynamic References	Use {{ }} notation. {{job.start_time.day}}, {{task.name}}, {{tasks.X.values.Y}}
Task Values API	dbutils.jobs.taskValues.set() to write. dbutils.jobs.taskValues.get() to read.
Repair & Run	Re-runs ONLY failed tasks. Does NOT fix the job definition permanently.
Serverless advantage	Photon default, no VM management, auto-scaling, shielded from cloud disruptions.
Interactive Cluster warning	NOT for production — too costly, limited scalability, shared availability issues.
Table Update Trigger	Fires when specified tables receive insert/update/delete/merge. Up to 10 source tables.
For Each downstream	Downstream tasks depend on the For Each CONTAINER, not individual iterations.
If/Else condition	"None of dependency failed AND at least one task executed" — evaluates boolean expressions.
Medallion layers	Bronze = raw. Silver = cleaned. Gold = business aggregations.
Delta log location	_delta_log/ folder stores JSON transaction logs alongside Parquet data files.
schema_of_json()	Auto-derives STRUCT schema from a sample JSON string — avoids manual schema writing.
from_json()	Converts a JSON STRING column to a typed STRUCT column using a provided schema.

VARIANT type	Public Preview 2025 Q2. Stores any data including JSON. More flexible and performant than STRING.
system.lakeflow	Built-in read-only catalog for job monitoring. Key tables: jobs, job_tasks, job_run_timeline.

Useful Source URLs

Lakeflow Connect (Auto Loader)
File Metadata Column (_metadata)
Databricks Release Notes / Preview Features
SaaS Managed Connectors
Database Managed Connectors
What is Databricks Partner Connect?
Lakehouse Federation
What is Delta Sharing?
Databricks Marketplace
Delta Lake Documentation
Lakeflow Jobs Documentation
Lakeflow Declarative Pipelines (DLT)
Stream Processing with Apache Spark Course
Build Data Pipelines with Lakeflow Declarative Pipelines Course
Announcing GA of Databricks Lakeflow

Study Guide generated from course slides — Introduction to Data Engineering in Databricks (Chapters 1–2)
© Databricks 2025. All course content rights reserved.